EVENT-DRIVEN ARCHITECTURE CHEAT SHEET

Core setup, initialization, operations & integration quick reference

PART 1: SETUP & CORE

01. OVERVIEW & SETUP

Installation

Event-Driven Architecture Architecture technology

```
# Install
npm install event-driven-architecture
# Or: pip install / gem install / go get
```

Check official documentation at docs.event-drivenarchitecture.com
Verify installation before proceeding

02. CORE CONCEPTS

Architecture

Key abstraction: understand the primary model
Configuration: define behavior through config/code
Integration: connects with existing systems

```
# Initialize
const client = new Event-DrivenArchitecture({...
```

03. QUICK START

YAML / Config

```
# Minimal working example
const app = new Event-DrivenArchitecture()
app.configure({ ... })
await app.start()
console.log('Event-Driven Architecture is run...
```

See official quickstart guide for language-specific setup

04. CONFIGURATION

Workflow

```
# config.yaml or config.json
event-driven:
host: localhost
port: 8080
timeout: 30s
```

Use environment variables for secrets

```
export EVENT-DRIVEN_ARCHITECTURE_SECRET=my-se...
```

05. BASIC OPERATIONS

CRUD API

```
# Create / write
client.create({ name: 'example' })
# Read / query
const result = await client.find({ filter: {}...
# Update
await client.update(id, { field: newValue })
# Delete
await client.delete(id)
```

06. INTEGRATION

Integration

Integrates with common frameworks and tools

```
# Express.js integration example
app.use(Event-DrivenArchitectureMiddleware({ ...
```

SDK available for: Node.js, Python, Java, Go, .NET
REST API available for language-agnostic integration
Check community-maintained integrations

SECURITY, MONITORING & OPERATIONS

Production hardening, observability, performance tuning & CLI reference

PART 2: OPS & SECURITY

07. SECURITY

Hardening

Always use authentication and authorization
Encrypt data at rest and in transit (TLS/SSL)

```
# Enable TLS
tls: { cert: ./cert.pem, key: ./key.pem }
```

Rotate credentials and API keys regularly
Follow security best practices in official docs

10. TESTING

Unit Tests

```
# Unit test example
describe('Event-Driven Architecture', () => {
it('should work', async () => {
const result = await client.doSomething()
expect(result).toBeDefined()
})
})
```

Use mocks for external dependencies in unit tests

08. MONITORING

Observability

Expose health check endpoint

```
# Health check
GET /health  { status: 'ok', uptime: 1234 }
```

Monitor key metrics: latency, throughput, error rate
Set up alerts for anomalies
Log requests with structured logging (JSON format)

11. CLI REFERENCE

Commands

```
event-driven --help       # all commands
event-driven version      # version info
event-driven config       # configuration
event-driven status       # current status
event-driven logs         # view logs
```

09. PERFORMANCE TIPS

Optimization

Use connection pooling for network resources
Enable caching for frequently accessed data
Batch operations to reduce round trips

```
# Async/parallel processing
await Promise.all([op1(), op2(), op3()])
```

Profile before optimizing measure, then improve

12. COMMON GOTCHAS

Warning

Read the changelog before upgrading major versions
Check resource limits (memory, connections, rate limits)
Implement graceful shutdown to avoid data loss
Keep dependencies updated for security patches
Test in staging environment before deploying to production